P R O J E C T

A Cross-Platform Case Study of OS Telemetry Evasion and Kernel-Resident Detection on Windows 11 and Ubuntu

Can an OS be made blind to itself — and can we still catch it?

CSC I0420 — Secure Operating Systems · City College of New York · Spring 2026

Presenter: Proma Roy

Presenting to: Joseph Soryal

MITRE T1562.006

CVE-2021-21551

Three Surprises

Motivation

Three real-world cases that shaped every design decision in this project.

STATE-SPONSORED APT

Lazarus Group FudModule Rootkit

North Korean state actor. Disabled EDR kernel callbacks from user space — no kernel exploit at runtime. Used BYOVD to obtain kernel access, then blinded security software without a crash.

Technique: BYOVD kernel callback removal · 2022

COMMERCIAL MALWARE

Remcos RAT

Commercially sold tool repurposed for attacks. Applies the EtwEventWrite 0xC3 patch in memory before running its payload — makes the process invisible to any EDR tool relying on ETW.

Technique: In-process ETW patch · Ongoing

CVE-2021-21551 · CVSS 8.8

Dell DBUtil Driver Privilege Escalation

Signed Dell driver on millions of machines. A kernel read/write flaw let Lazarus obtain Ring 0 access before deploying FudModule — the escalation step that always precedes telemetry silencing.

Local Privilege Escalation · Patch: May 2021

MITRE ATT&CK · T1562.006 · TA0005 Defense Evasion · Impair Defenses: Disable or Modify ETW / Linux Audit System

Three Environments Setup - The Windows VM, the Hardest Part

MOST DIFFICULT

Windows 11 PC

Host · Development Machine

VS2022 + EWDK
Kernel driver dev environment

Python scripts
ETW monitor + correlator

WinDbg (host side)
Sends debug over VMnet8

etw_tamper.exe
Compiled, deployed to VM

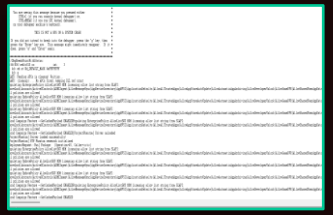
WinDbg debug →

Windows 11 VM

Target Machine · VMware · Test-signed kernel mode

Where PhantomGuard.sys loads · Where ETW gets patched · Where the BSODs happened

WinDbg → VM connected



Driver loaded successfully

PhantomGuard.sys
Kernel callback — independent stream

etw_tamper.exe
0xC3 patch → ETW silenced

streams →

Ubuntu 24.04 VM

VirtualBox · kernel 6.17 · Linux side

auditd monitor
execve · phantom_exec tag

phantom_guard.bt
eBPF (Extended Berkeley Packet Filter) tracepoint — always on

auditctl -e 0
Tamper — instant blind spot

May 5, 2026
12:56:14 blind spot confirmed

cross_correlator.py

2-second sliding window · compares Windows VM + Ubuntu streams · flags divergence as a JSON SIEM alert the moment they disagree

→ ALERT

What Was Actually Built - Windows + Linux

Windows 11 — Six Phases

01 — Hello World Kernel Driver
Confirmed VS2022 + EWDK + WinDbg over VMnet8 — toolchain proven before anything else.

02 — Python ETW Baseline Monitor
Microsoft-Windows-Kernel-Process provider, JSON log per process creation event.

03 — C++ ETW Tamper Tool (etw_tamper.cpp)
0xC3 RET patch at EtwEventWrite byte zero in ntdll.dll — ETW silenced in the target process.

04 — PhantomGuard.sys Kernel Driver
PsSetCreateProcessNotifyRoutineEx — independent stream the patch cannot reach.

05 — Python Cross-Correlator
2-second sliding window, phantom_alerts.json, T1562.006 attribution in every alert.

06 — A Test Evaluation Suite
Automated test runner + timing side-channel analysis across tampered and clean runs.

Ubuntu VM — Linux Extension

- **audit_monitor.py v5**
  tail -f on /var/log/audit/audit.log
- **phantom_guard.bt**
  bpftrace eBPF · execve tracepoint
- **Tamper: auditctl -e 0**
  Root cmd · instant system-wide silence
- **Live demo**
  12:56:14, May 5, blind spot confirmed in real time

Tools

C++ · Visual Studio 2022 Community

WDK 11 · EWDK · WinDbg Preview

Python 3.11 · pywin32

bpftrace · eBPF · auditd

VirtualBox · VMware (debug target)

Observation #1 - Windows Fought Back

CASE 01

The Attack

01 **Located EtwEventWrite in ntdll.dll**
The function Windows uses to send every security event into the ETW pipeline

02 **Changed one byte in memory**

BEFORE		AFTER
`0x40 PUSH rax`	⇒	`0xC3 RET`

03 **Expected: complete silence**
ETW stream goes dark — no events logged from affected process

! **Got: Error 0x00000027 (ERROR_NOT_READY)**
Undocumented internal ETW integrity check — never seen in any public reference

Why Windows Could Resist

RING 0 — KERNEL

ETW dispatch table lives here
Lazarus Group FudModule operates here
Requires signed driver + Microsoft authorization

PROTECTED BOUNDARY — Cannot be crossed from Ring 3

RING 3 — USER MODE

Where I operated
ntdll.dll patch applied here
Windows detected it and returned 0x27

First surprise:

Windows 11 is actively defending its telemetry pipeline in ways that are not publicly documented. The 0x27 response appeared on every tampered run — consistently, reproducibly.

Observation #2 - The BSOD

CASE
02

Windows 11 PC

Host · Dev machine

- VS2022 + EWDK
- WinDbg (sends debug to VM)
- Python scripts run here

Windows 11 VM

MOST DIFFICULT

TARGET · Most painful

- Driver loaded here
- ETW patched here
- BSODs happened here

Ubuntu 24.04 VM

Linux side · VirtualBox

- auditd monitor
- phantom_guard.bt (eBPF)
- Linux extension

The BSOD — DRIVER_IRQL_NOT

Cause: wrong callback signature

PsSetCreateProcessNotifyRoutine
vs
PsSetCreateProcessNotifyRoutineEx

One mismatched parameter type. No warning. No error message. The entire OS goes down.

WinDbg → VM connected

After fix — PhantomGuard working

Driver loaded successfully

Captures every process on the system
Completely independent of ETW
Unaffected by the 0xC3 patch

Second surprise:

The most important lesson in kernel development is not how to write the code. It is how to survive getting it wrong.

Observation #3 — The Inverse Timing

CASE 03

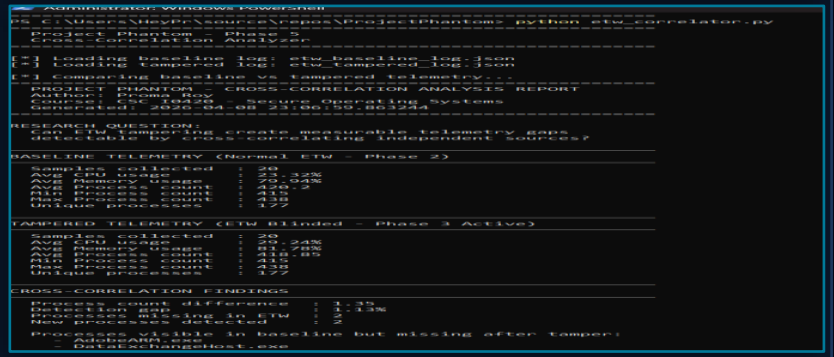
Cross-Correlation Results

1.13%
detection gap

0
false positives (50 tests)

Ghost Processes — ETW-invisible without tampering

`AdobeARM.exe` · `DataExchangeHost.exe`



Actual cross-correlator output · April 8, 2026

The Timing Finding

Expected

Tampering → more chaos → ↑ jitter
Scheduling becomes unstable, irregular

VS

Actual

Tampering → less work → ↓ jitter
0xC3 skips all logging — fewer cycles used, tighter scheduling

Third surprise:

Blinding the OS made it more efficient. That efficiency change is a timing signature — detectable from user space without any kernel privileges at all.

A new detection path nobody else has documented.

What I Proved — Windows and Linux

Windows — Partial Blind

1.13% detection gap

2 processes hidden from ETW at Ring 3

Ring 3 limitation

Could not achieve full blind from user mode

ETW integrity check

0x27 error — OS detected the tamper attempt

Kernel sensor: 100%

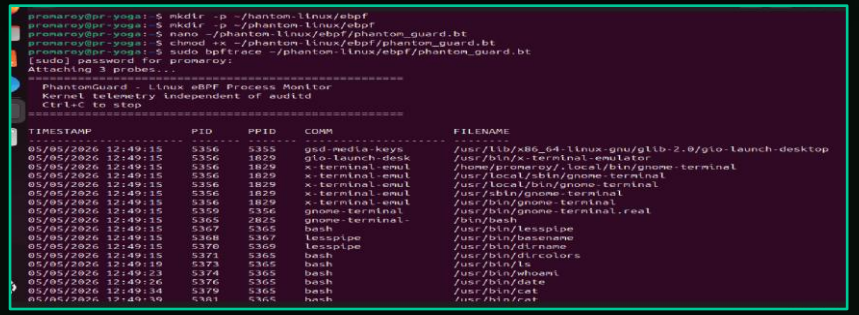
PhantomGuard saw everything ETW missed

12/12 tests passed · 0 false positives across 50 benign launches

Linux — Total Blind

`auditctl -e 0` → Total silence

Not 1.13%. Every audit record stopped instantly, system-wide



phantom_guard.bt — eBPF output · May 5, 2026

Blind spot: 12:56:14 → 12:58:52 = 2 min 38 sec

eBPF: 100% CAPTURE THROUGH ENTIRE BLIND SPOT

Different OS. Different kernel. Same architectural result: the independent kernel sensor saw everything the primary stream missed.

What I Couldn't Do - and Future Work

The Honest Result — Windows

Could not achieve full blind from Ring 3

1.13% gap — not 100%. The Ring 3 / Ring 0 boundary held.

What a full blind requires

Signed kernel driver with Ring 0 access
Microsoft EV code signing authorization
ETW internal dispatch table manipulation
→ Exactly what Lazarus Group has. I don't.

The gap between 1.13% and 100%

is exactly where the most interesting security research lives.

What's Next — macOS (Summer 2026)

Attempted briefly during this project. Hit a wall immediately — macOS uses the Endpoint Security framework, SIP, and strict codesigning. It is a completely different research problem that cannot be approached the same way as Windows or Linux.

Planned: Endpoint Security API · macOS eBPF equivalent
Dedicated project — not an afterthought

Full Code GitHub

`PhantomGuard.sys` · `etw_tamper.cpp`
`cross_correlator.py` · 12-test suite
`audit_monitor.py` · `phantom_guard.bt`

https://github.com/HeyPromaRoy/Secure-Operating-system.git